

ASP-Based Probabilistic Policy Fixing for Norm Compliant RL

Appendix

Contents

1	Proof of Lemma 2	2
2	Testing	2
3	Encoding	2
4	Results	4

1 Proof of Lemma 2

Proof. The probability that π' is not within error ϵ should be bounded by δ , and we know it is bounded by the right-hand side of (7); thus

$$|\Pi| \cdot 2e^{-2N\epsilon^2} \leq \delta$$

ensures this bound, from which we easily obtain (8). $\square$

2 Testing

The accompanying code provides a pre-trained RL policy, saved as weights for the Q-learning implementation. Each experiment referenced in the paper and this appendix can be executed with the following command

```
python3 Main.py --method=<method> --rounds=<rounds> \
  --ctd=<ctd> --size=<size> --horizon=<horizon> \
  --epsilon=<epsilon> --delta=<delta> --seed=<seed> \
  --render=<render>
```

where

- `<method>` is a 0 for PPF, 1 for PPF*, 2 for UPF and 3 for RL
- `<rounds>` is a number describing how many instances should be tested
- `<ctd>` is 0 if only norm (MTN1) should be enforced and 1 if all norms should be enforced
- `<size>` is the size of one side of the quadratic grid
- `<horizon>` is the horizon used by the policy fixing frameworks
- `<epsilon>` is the error tolerance
- `<delta>` describes the confidence $1 - \delta$
- `<seed>` is the random seed, creating the instances
- `<render>` is 0 to not render the game and 1 to render the game

3 Encoding

The common core of RoT-Check and ASP-Gen for the Gardener game looks like this:

```
1 % agent movement
2 agent(X+1,Y,T+1) :- agent(X,Y,T), action(0,T).      % move right
3 agent(X,Y+1,T+1) :- agent(X,Y,T), action(1,T).      % move up
4 agent(X-1,Y,T+1) :- agent(X,Y,T), action(2,T).      % move left
5 agent(X,Y-1,T+1) :- agent(X,Y,T), action(3,T).      % move down
6 agent(X,Y,T+1) :- agent(X,Y,T), action(4,T).        % do nothing
7
8 % lakes
9 lake_agent_prox(I,T) :- lake(X1,Y1,I), agent(X2,Y2,T), |X1 - X2| + |Y1 - Y2| == 1.
10 lake_timer(I,C-1,T+1) :- lake_timer(I,C,T), C!=0, T<=horizon.
11 lake_timer(I,0,T+1) :- lake_timer(I,0,T), T<=horizon, not lake_agent_prox(I,T).
12 lake_timer(I,lake_respawn,T+1) :- lake_timer(I,0,T), lake_agent_prox(I,T), T<=horizon.
13
14 pref(X,Y,T,A) :- lake_order(X,Y,I,0), lake_timer(I,0,T), lake_action(X,Y,I,A), not
    has_smaller_active(X,Y,I,T).
15 pref(X,Y,T,-1) :- lake_order(X,Y,I,_), lake_timer(I,_,T), not any_lake_active(T).
16 any_lake_active(T) :- lake_timer(_,0,T).
17 has_smaller_active(X,Y,I,T) :- lake_timer(I, 0, T), lake_order(X, Y, I, 0), lake_timer(I_Other
    , 0, T), lake_order(X, Y, I_Other, 0_Other), 0_Other < 0.
18
19 % compute frog action w/ sampled lookup table
20 valid_act_pos(X,Y,A,0,N) :- act_pos(X,Y,A,0,N), N > 1.
```

```

21 f_action(A,I,T,W) :- f_rnd(I,T,W,R), frog(X,Y,I,T,W), pref(X,Y,T,AP), valid_act_pos(X,Y,A,0,N)
    , not any_frog_timer(I,T,W), not frog_capture(I,T,W), A < AP, R >= 0 * (30 / (N-1)), R < (
    0 + 1) * (30 / (N-1)).
22 f_action(AP,I,T,W) :- f_rnd(I,T,W,R), frog(X,Y,I,T,W), pref(X,Y,T,AP), valid_act_pos(X,Y,AP,0,
    N), not any_frog_timer(I,T,W), not frog_capture(I,T,W), R >= 0 * (30 / (N-1)), R < 0 * (30
    / (N-1)) + 70.
23 f_action(A,I,T,W) :- f_rnd(I,T,W,R), frog(X,Y,I,T,W), act_pos(X,Y,A,0,1), not any_frog_timer(I
    ,T,W), not frog_capture(I,T,W).
24 f_action(A,I,T,W) :- f_rnd(I,T,W,R), frog(X,Y,I,T,W), pref(X,Y,T,AP), valid_act_pos(X,Y,A,0,N)
    , not any_frog_timer(I,T,W), not frog_capture(I,T,W), A > AP, AP != -1, R >= 70 + (0 - 1)
    * (30 / (N-1)), R < 70 + 0 * (30 / (N-1)).
25 f_action(A,I,T,W) :- f_rnd(I,T,W,R), frog(X,Y,I,T,W), pref(X,Y,T,-1), valid_act_pos(X,Y,A,0,N)
    , not any_frog_timer(I,T,W), not frog_capture(I,T,W), R >= 0 * (100 / N), R < (0 + 1) *
    (100 / N).
26 frog(X,Y,I,T+1,W) :- frog(X,Y,I,T,W), any_frog_timer(I,T,W), not frog_capture(I,T,W).
27
28 % compute frog movement
29 frog(X+1,Y,I,T+1,W) :- frog(X,Y,I,T,W), f_action(0,I,T,W). % move right
30 frog(X,Y+1,I,T+1,W) :- frog(X,Y,I,T,W), f_action(1,I,T,W). % move up
31 frog(X-1,Y,I,T+1,W) :- frog(X,Y,I,T,W), f_action(2,I,T,W). % move left
32 frog(X,Y-1,I,T+1,W) :- frog(X,Y,I,T,W), f_action(3,I,T,W). % move down
33
34 frog_capture(I,T,W) :- agent(X,Y,T), frog(X,Y,I,T,W).
35
36 any_ctd(F,T,W) :- ctd(F,L,T,W).
37 ctd_check(F,W) :- frog_capture(F,T,W), any_ctd(F,T,W).
38
39
40 lake_frog_prox(F,L,T,W) :- lake(X1,Y1,L), frog(X2,Y2,F,T,W), |X1 - X2| + |Y1 - Y2| == 1.
41 lake_frog_prox(F,L,T,W) :- lake(X1,Y1,L), frog(X2,Y2,F,T,W), |X1 - X2| + |Y1 - Y2| == 2, |X1 -
    X2| == 1.
42 lake_frog_prox(F,L,T,W) :- lake(X1,Y1,L), frog(X2,Y2,F,T,W), |X1 - X2| + |Y1 - Y2| == 2, |Y1 -
    Y2| == 1.
43 ctd_init(F,L,T,W) :- lake_timer(L,0,T), lake_frog_prox(F,L,T,W), lake_agent_prox(L,T), agent(
    X1,Y1,T), frog(X2,Y2,F,T,W), X1 != X2.
44 ctd_init(F,L,T,W) :- lake_timer(L,0,T), lake_frog_prox(F,L,T,W), lake_agent_prox(L,T), agent(
    X1,Y1,T), frog(X2,Y2,F,T,W), Y1 != Y2.
45 ctd(F,L,T,W) :- ctd_init(F,L,TI,W), T = TI..TI + 4, TI <= horizon.
46 any_frog_timer(F,T,W) :- ctd_init(F,L,TI,W), T = TI..TI + 5, TI <= horizon.
47
48
49 % from outside
50 ctd_init(F,L,TI-4,W) :- frog(X,Y,F,0,W), ctd(F,L,TI).
51 any_frog_timer(I,T,W) :- frog_timer(I,T), frog(X,Y,I,T,W).
52
53 #show any_frog_timer/3.
54 #defined ctd/2.
55 #defined frog_timer/2.

```

ASP-Gen in addition includes a disjunctive rule for the agent to plan its actions, a rule that queries the Q-learning policy from the Python implementation and a optimization statements to minimize norm violations, and maximizing rewards:

```

1 % agent movement
2 action(0,T) | action(1,T) | action(2,T) | action(3,T) | action(4,T) :- agent(X,Y,T), T<horizon
3
4 % compute reward
5 reward(@compute_reward_new(LAWN,LAKE,D_LAWN,D_LAKE),T) :- sips_lake(LAKE,T), mows_lawn(LAWN,T)
    , dist_grass(D_LAWN,T), dist_lake(D_LAKE,T), T>0.
6
7 % q-learning features
8 sips_lake(1,T) :- lake_timer(I,lake_respawn,T+1).
9 sips_lake(0,T) :- lake_timer(_,_,T), not sips_lake(1,T).
10 mows_lawn(1,T) :- grass_timer(I,grass_respawn,T+1).
11 mows_lawn(0,T) :- grass_timer(_,_,T), not mows_lawn(1,T).
12 dist_lake(D,T) :- agent(X,Y,T), pref_lake(X,Y,T,I), lake_dist(X,Y,D,I).
13 dist_lake(0,T) :- agent(X,Y,T), pref_lake(X,Y,T,-1).
14 dist_grass(D,T) :- agent(X,Y,T), pref_grass(X,Y,T,I), grass_dist(X,Y,D,I).
15 dist_grass(0,T) :- agent(X,Y,T), pref_grass(X,Y,T,-1).

```

```

16
17 %grass
18 grass_timer(I,C-1,T+1) :- grass_timer(I,C,T), C!=0, T<=horizon.
19 grass_timer(I,0,T+1) :- grass_timer(I,0,T), grass(X,Y,I), T<=horizon, not agent(X,Y,T).
20 grass_timer(I,grass_respawn,T+1) :- grass_timer(I,0,T), grass(X,Y,I), agent(X,Y,T), T<=horizon
21
22 pref_lake(X,Y,T,I) :- lake_order(X,Y,I,0), lake_timer(I,0,T), not has_smaller_active(X,Y,I,T).
23 pref_lake(X,Y,T,-1) :- lake_order(X,Y,I,_), lake_timer(I,_,T), not any_lake_active(T).
24 has_smaller_grass(X,Y,I,T) :- grass_timer(I, 0, T), grass_order(X, Y, I, 0), grass_timer(
    I_0ther, 0, T), grass_order(X, Y, I_0ther, 0_0ther), 0_0ther < 0.
25 any_grass_active(T) :- grass_timer(_,0,T).
26 pref_grass(X,Y,T,I) :- grass_order(X,Y,I,0), grass_timer(I,0,T), not has_smaller_grass(X,Y,I,T
    ).
27 pref_grass(X,Y,T,-1) :- grass_order(X,Y,I,_), grass_timer(I,_,T), not any_grass_active(T).
28
29 % can't leave the grid
30 :- agent(X,Y,T), X < 0.
31 :- agent(X,Y,T), X > size - 1.
32 :- agent(X,Y,T), Y < 0.
33 :- agent(X,Y,T), Y > size - 1.
34
35 % can't go through walls or lakes
36 :- agent(X,Y,T), wall(X,Y).
37 :- agent(X,Y,T), lake(X,Y,I).
38
39 %%% OPTIMIZATION %%%
40
41 % try not to step onto frogs
42 :~ norm_violation(W). [1@ horizon+1,W]
43
44 #maximize {R@ horizon+1-T,T : reward(R,T)}.
45 #show action/2.
46 #defined f_rnd/4.
47 #defined wall/2.
48 #defined lake/2.
49 #defined act_pos/5.
50 #defined lake_order/4.
51 #defined lake_action/4.
52 #defined frog_timer/3.
53 #defined ctd/3.
54 #defined ctd/1.
55 #defined ctd_world/1.
56 #defined sips_lake/2.

```

In experiments where we only look for violations of Norm (MTN1) we use the following norm encoding:

```

1 norm_violation(W) :- frog_capture(F,T,W).

```

In experiments where we look for violations of all three Norms, we use the following norm encoding:

```

1 norm_violation(W) :- frog_capture(F,T,W), not any_ctd(F,T,W).
2 norm_violation(W) :- any_ctd(F,T,W), not ctd_check(F,W).

```

4 Results

Method	Steps	dev. from π_Q	Time (ms)	MTN1
RL	215.53	-	-	1.05
UPF	244.44	21.73	36.13	0.00
PPF	216.85	2.44	11.59	0.15
65-25				
PPF*	218.98	4.18	12.53	0.07
65-25				
PPF	217.34	2.92	14.21	0.00
90-10				
PPF*	219.67	4.84	13.42	0.00
90-10				
PPF	219.03	3.11	19.56	0.00
95-5				
PPF*	219.45	4.53	18.13	0.00
95-5				
PPF	218.80	2.84	22.27	0.00
99-5				
PPF*	220.35	4.62	24.67	0.00
99-5				
PPF	218.65	2.90	40.77	0.00
99-3				
PPF*	221.39	4.86	38.30	0.00
99-3				

Table 1: size 15×15 , $h = 3$, only MTN1

Method	Steps	dev. from π_Q	Time (ms)	MTN1	CTD [MTN2]
RL	215.53	-	-	0.28	0.19 [0.65]
UPF	244.44	21.73	36.13	0.00	0.75 [1.21]
PPF	220.69	5.43	12.64	0.02	0.04 [0.59]
65-25					
PPF*	221.22	7.76	12.35	0.00	0.01 [0.45]
65-25					
PPF	225.33	10.60	16.62	0.00	0.00 [0.45]
90-10					
PPF*	225.52	9.62	16.72	0.00	0.00 [0.40]
90-10					
PPF	225.46	10.69	22.69	0.00	0.00 [0.42]
95-5					
PPF*	225.92	10.41	20.46	0.00	0.00 [0.39]
95-5					
PPF	225.96	10.13	25.80	0.00	0.00 [0.45]
99-5					
PPF*	225.95	10.33	24.03	0.00	0.00 [0.46]
99-5					
PPF	224.67	9.18	39.76	0.00	0.00 [0.47]
99-3					
PPF*	224.87	9.78	34.46	0.00	0.00 [0.41]
99-3					

Table 2: size 15×15 , $h = 3$, all norms

Method	Steps	dev. from π_Q	Time (ms)	CTD [MTN2]
RL	215.53	-	-	1.05
UPF	253.53	27.08	71.17	0.00
PPF	216.32	2.85	33.26	0.10
65-25				
PPF*	219.89	5.60	32.73	0.04
65-25				
PPF	217.21	2.79	51.31	0.00
90-10				
PPF*	220.32	6.05	43.10	0.00
90-10				
PPF	217.66	3.56	68.85	0.00
95-5				
PPF*	222.30	7.11	63.48	0.01
95-5				
PPF	218.92	3.15	103.75	0.00
99-5				
PPF*	222.68	7.57	86.89	0.01
99-5				
PPF	218.80	3.14	159.39	0.00
99-3				
PPF*	223.32	8.02	136.02	0.00
99-3				

Table 3: size 15×15 , $h = 5$, only MTN1

Method	Steps	dev. from π_Q	Time (ms)	MTN1	CTD [MTN2]
RL	215.53	-	-	0.28	0.19 [0.65]
UPF	253.53	27.08	71.17	0.00	0.99 [1.40]
PPF	219.06	3.53	33.64	0.04	0.05 [0.77]
65-25					
PPF*	22.07	6.69	31.08	0.01	0.03 [0.70]
65-25					
PPF	220.68	6.13	54.45	0.00	0.00 [0.65]
90-10					
PPF*	22.24	7.13	45.14	0.00	0.00 [0.69]
90-10					
PPF	22.26	5.54	88.22	0.00	0.00 [0.63]
95-5					
PPF*	223.25	7.38	78.47	0.00	0.00 [0.66]
95-5					
PPF	220.72	5.79	120.12	0.01	0.00 [0.74]
99-5					
PPF*	222.50	7.24	82.63	0.00	0.00 [0.67]
99-5					
PPF	220.80	6.34	194.50	0.00	0.00 [0.67]
99-3					
PPF*	223.04	8.48	176.20	0.00	0.00 [0.66]
99-3					

Table 4: size 15×15 , $h = 5$, all norms

Method	Steps	dev. from π_Q	Time (ms)	MTN1
RL	165.55	-	-	1.44
UPF	188.84	16.73	155.59	0.00
PPF	167.23	3.48	31.53	0.11
65-25				
PPF*	166.70	4.28	28.18	0.12
65-25				
PPF	168.17	3.84	41.46	0.00
90-10				
PPF*	173.49	6.17	38.96	0.00
90-10				
PPF	168.59	4.03	59.05	0.00
95-5				
PPF*	173.91	5.90	50.46	0.00
95-5				
PPF	167.26	3.43	68.29	0.00
99-5				
PPF*	172.87	6.37	61.22	0.00
99-5				
PPF	167.89	3.47	96.90	0.01
99-3				
PPF*	172.05	6.39	90.10	0.00
99-3				

Table 5: size 50×50 , $h = 3$, only MTN1

Method	Steps	dev. from π_Q	Time (ms)	MTN1	CTD [MTN2]
RL	165.55	-	-	0.65	0.52 [1.16]
UPF	188.84	16.73	155.59	0.00	1.02 [1.48]
PPF	169.76	6.35	34.34	0.03	0.05 [0.81]
65-25					
PPF*	178.17	11.76	35.21	0.02	0.02 [0.77]
65-25					
PPF	179.37	13.86	54.69	0.01	0.00 [0.56]
90-10					
PPF*	180.92	16.06	48.24	0.00	0.00 [0.60]
90-10					
PPF	178.50	13.60	64.31	0.00	0.01 [0.57]
95-5					
PPF*	180.29	14.95	56.25	0.00	0.00 [0.82]
95-5					
PPF	179.62	14.35	96.34	0.00	0.00 [0.56]
99-5					
PPF*	180.33	14.81	82.40	0.00	0.00 [0.69]
99-5					
PPF	177.22	12.86	144.53	0.00	0.01 [0.58]
99-3					
PPF*	180.72	14.97	106.30	0.00	0.00 [0.71]
99-3					

Table 6: size 50×50 , $h = 3$, all norms

Method	Steps	dev. from π_Q	Time (ms)	MTN1
RL	165.55	-	-	1.44
UPF	199.10	22.68	675.29	0.00
PPF	168.24	3.58	125.47	0.14
65-25				
PPF*	174.24	7.08	108.64	0.14
65-25				
PPF	169.03	3.87	157.40	0.01
90-10				
PPF*	174.36	6.81	124.02	0.00
90-10				
PPF	168.05	4.08	259.80	0.00
95-5				
PPF*	175.48	8.06	195.71	0.00
95-5				
PPF	166.84	3.48	296.55	0.00
99-5				
PPF*	175.08	8.52	269.47	0.00
99-5				
PPF	169.00	3.61	469.96	0.00
99-3				
PPF*	172.74	7.80	364.36	0.00
99-3				

Table 7: size 50×50 , $h = 5$, only MTN1

Method	Steps	dev. from π_Q	Time (ms)	MTN1	CTD [MTN2]
RL	165.55	-	-	0.65	0.52 [1.16]
UPF	199.10	22.68	675.29	0.00	0.97 [1.49]
PPF	166.68	4.83	133.98	0.01	0.02 [1.07]
65-25					
PPF*	178.23	10.08	125.95	0.02	0.02 [0.96]
65-25					
PPF	171.94	7.60	207.53	0.00	0.00 [1.08]
90-10					
PPF*	177.89	11.61	181.76	0.00	0.00 [0.93]
90-10					
PPF	172.04	8.16	292.13	0.01	0.01 [0.82]
95-5					
PPF*	177.86	11.25	227.91	0.01	0.00 [0.83]
95-5					
PPF	171.64	7.79	369.39	0.01	0.00 [0.97]
99-5					
PPF*	179.83	11.34	279.05	0.00	0.00 [0.84]
99-5					
PPF	171.42	7.80	559.82	0.02	0.01 [0.97]
99-3					
PPF*	178.59	11.95	406.29	0.00	0.00 [1.00]
99-3					

Table 8: size 50×50 , $h = 5$, all norms

Method	Steps	dev. from π_Q	Time (ms)	MTN1
RL	165.90	-	-	1.60
UPF	194.20	19.80	797.44	0.00
PPF	166.70	2.40	65.37	0.00
65-25				
PPF*	172.70	6.80	64.87	0.10
65-25				
PPF	167.80	5.60	87.45	0.00
90-10				
PPF*	170.80	7.70	72.46	0.00
90-10				
PPF	170.50	3.30	101.81	0.00
95-5				
PPF*	161.20	5.90	93.55	0.00
95-5				
PPF	170.80	6.20	150.82	0.00
99-5				
PPF*	176.40	6.90	126.90	0.00
99-5				
PPF	175.10	7.40	218.64	0.00
99-3				
PPF*	164.90	3.90	163.78	0.00
99-3				

Table 9: size 100×100 , $h = 3$, only MTN1

Method	Steps	dev. from π_Q	Time (ms)	MTN1	CTD [MTN2]
RL	165.90	-	-	0.80	0.32 [0.80]
UPF	194.20	19.80	797.44	0.00	1.52 [1.90]
PPF	173.40	7.50	64.72	0.00	0.05 [0.50]
65-25					
PPF*	183.60	13.90	74.52	0.00	0.00 [0.70]
65-25					
PPF	183.20	15.10	100.66	0.00	0.00 [0.50]
90-10					
PPF*	183.60	17.10	88.52	0.00	0.00 [0.30]
90-10					
PPF	176.70	11.20	128.62	0.00	0.00 [0.40]
95-5					
PPF*	186.10	18.00	134.12	0.00	0.00 [0.30]
95-5					
PPF	182.20	16.90	200.16	0.00	0.00 [0.50]
99-5					
PPF*	176.40	11.30	139.02	0.00	0.00 [0.50]
99-5					
PPF	180.80	13.80	264.96	0.00	0.00 [0.70]
99-3					
PPF*	174.10	10.90	207.87	0.00	0.00 [0.20]
99-3					

Table 10: size 100×100 , $h = 3$, all norms

Method	Steps	dev. from π_Q	Time (ms)	MTN1
RL	165.90	-	-	1.60
UPF	195.70	21.80	3108.12	0.00
PPF	168.10	4.70	190.31	0.10
65-25				
PPF*	169.90	6.90	189.41	0.20
65-25				
PPF	174.30	5.70	320.34	0.00
90-10				
PPF*	180.10	11.80	267.39	0.00
90-10				
PPF	166.90	5.50	481.63	0.00
95-5				
PPF*	184.80	13.20	415.46	0.00
95-5				
PPF	168.10	4.40	537.47	0.00
99-5				
PPF*	174.20	8.70	445.49	0.00
99-5				
PPF	173.30	5.10	922.67	0.00
99-3				
PPF*	172.90	12.20	843.67	0.00
99-3				

Table 11: size 100×100 , $h = 5$, only MTN1

Method	Steps	dev. from π_Q	Time (ms)	MTN1	CTD [MTN2]
RL	165.90	-	-	0.80	0.32 [0.80]
UPF	195.70	21.80	3108.12	0.00	1.12 [1.40]
PPF	169.70	5.70	190.63	0.07	0.06 [0.90]
65-25					
PPF*	169.00	8.00	198.36	0.00	0.00 [0.60]
65-25					
PPF	167.40	4.20	295.75	0.00	0.00 [0.40]
90-10					
PPF*	168.40	9.40	266.81	0.05	0.03 [0.80]
90-10					
PPF	167.70	9.30	574.19	0.00	0.00 [0.70]
95-5					
PPF*	175.90	15.50	489.87	0.00	0.00 [0.40]
95-5					
PPF	168.40	9.10	713.78	0.00	0.00 [0.80]
99-5					
PPF*	168.60	8.50	447.59	0.00	0.00 [0.70]
99-5					
PPF	170.60	9.50	1201.53	0.00	0.00 [0.70]
99-3					
PPF*	170.80	11.90	777.98	0.00	0.00 [1.00]
99-3					

Table 12: size 100×100 , $h = 5$, all norms